

Week 12: Graded Mini Project

This **Graded Project** asks you to design and implement a **goal-oriented multi-agent workflow** using **CrewAI**, inspired by the *Customer Support multi-agent flow* demonstrated in class (Week 11), but applied to a new industry domain. The focus is on your ability to define clear agent roles, coordinate handoffs, and implement decision-making and escalation logic in a realistic workflow.

Note: Retrieval-Augmented Generation (RAG) is **not required**. Knowledge sources may be simulated using static context, mock data, predefined assumptions, hardcoded rules, or static policy text.

Where to find the tasks

The complete project brief, starter structure, and submission environment will be available under **Week 12 – Graded Project** in [Vocareum](#). Follow the instructions in the notebook/environment to implement your multi-agent workflow, run sample test inputs, and document your design decisions.

What Was Covered in Class (Reference)

In class, we implemented a **Customer Support Goal-Oriented Multi-Agent Flow**, including:

- Issue Identification Agent
- Knowledge Search Agent (non-RAG, logic-based or static source)
- Solution Generation Agent
- Escalation Decision Agent

Project Objective

Your objective is to replicate the **architectural thinking** of a goal-oriented multi-agent system (not the exact same use case). You will build a workflow with multiple collaborating agents that can handle realistic user queries, apply rules/policies, and decide when to escalate.

Mandatory Architecture Requirements

- Your solution must include **at least 3–4 collaborating agents**.
- Each agent must have a **clear role** and a **specific goal**.
- Agents must show **dependency on previous agent outputs** (handoffs), not isolated responses.
- Your workflow must include **escalation logic** based on risk, urgency, or uncertainty.
- Include **sample user inputs** to demonstrate the end-to-end flow with outputs.

Choose ONE Industry Use Case

Use Case 1: Banking & Financial Services – Intelligent Customer Resolution

Problem Statement: Banks receive customer messages related to transactions, failed payments, card issues, suspicious activity, and loan inquiries. Responses must be accurate, cautious, and sometimes escalated.

Suggested Agents:

- Intent Classification Agent (fraud, transaction, loan, general)
- Banking Rules & Policy Reasoning Agent
- Response Drafting Agent
- Risk & Escalation Agent

Goal: Resolve routine queries automatically while escalating high-risk or fraud-related issues.

Use Case 2: E-Commerce – Order, Refund & Exception Handling

Problem Statement: E-commerce platforms handle large volumes of customer queries about delayed deliveries, refunds, damaged products, and return eligibility.

Suggested Agents:

- Order Issue Identification Agent
- Policy Interpretation Agent (returns, refunds, exceptions)
- Resolution Recommendation Agent
- Escalation Agent (manual approval required or not)

Goal: Provide fast resolutions while enforcing business policies consistently.

Use Case 3: HR Operations – Employee Support & Policy Guidance

Problem Statement: Employees raise questions about leave, payroll discrepancies, benefits, internal transfers, and grievances. Some queries are sensitive and must be escalated.

Suggested Agents:

- Query Classification Agent
- HR Policy Reasoning Agent
- Response Generation Agent
- Human HR Escalation Agent

Goal: Improve employee experience while protecting confidentiality and handling sensitive cases correctly.

Use Case 4: SaaS Product Support – Technical Issue Diagnosis

Problem Statement: SaaS companies receive technical queries related to login failures, feature usage, API errors, and integration issues.

Suggested Agents:

- Technical Issue Diagnosis Agent
- Product Knowledge Reasoning Agent (non-RAG)
- Troubleshooting Response Agent
- Engineering Escalation Agent

Goal: Reduce support workload while ensuring unresolved issues reach engineering teams.

Use Case 5: Supply Chain & Operations – Incident Management

Problem Statement: Operations teams receive incident reports related to shipment delays, inventory mismatches, vendor issues, or process failures.

Suggested Agents:

- Incident Classification Agent
- Operations Rules & SLA Reasoning Agent
- Action Recommendation Agent
- Escalation & Priority Decision Agent

Goal: Ensure timely response and correct prioritization of operational incidents.

Deliverables

- **CrewAI Code Implementation** (working multi-agent flow).
- **Architecture Explanation (1–2 pages)**, including:
 - Agent roles
 - Task flow / handoffs
 - Escalation logic
 - Sample inputs & outputs
 - Design rationale

How to Approach the Project

Start by selecting **one** industry use case and writing down 6–10 representative user queries that your system should handle (include at least a couple of “edge cases” that should trigger escalation).

Next, design your agent team:

- **Define each agent’s role + goal + inputs + outputs.** Keep the responsibilities tight (no “do-everything” agents).
- **Define the collaboration pattern** (sequential or conditional handoffs) so each agent depends on prior outputs.
- **Design escalation rules** (risk/uncertainty/urgency thresholds). Make escalation a deliberate decision, not a vague “if needed”.

Implement your flow in CrewAI and test it with your sample inputs. Use your tests to refine:

- handoff formats (what exactly gets passed forward),
- policy/rule clarity,
- escalation decisions (false positives/false negatives).

Finally, write your 1–2 page architecture explanation with diagrams or structured bullets: reviewers should be able to understand the workflow without reading every line of code.